Reviewer Pack — Schur Positivity of SOS Moment Matrices for 3-SAT

Entry aa14d6657f34 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 17:21:08 UTC

Schur Positivity of SOS Moment Matrices for 3-SAT

Entry ID: `aa14d6657f34` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 17:21:08 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Sum-of-Squares Hierarchy

Statement:

> For every 3-SAT instance with n variables, the SOS moment matrix of its solution space is Schur-positive iff the instance is satisfiable. The Schur positivity is quantified by the dominance of the first non-zero coefficient in the homogeneous symmetric function expansion.

Rationale (proposer's reasoning):

> Schur positivity captures combinatorial symmetry in symmetric functions, which may reflect hidden structure in the solution space's moment matrix. SOS hierarchies encode convex relaxations of CSPs, and their positivity properties could reveal algebraic constraints on satisfiability.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d495690207594d90`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3sat_instance(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, n) for _ in range(3)]
            clauses.append(clause)
        return clauses

    def is_satisfiable(clauses):
        n = max(abs(x) for x in sum(clauses, []))
        assignment = {var: random.choice([-1, 1]) for var in range(-n, n + 1)}
        for clause in clauses:
            if not any(assignment[var] * x <= 0 for x in clause):
                return True
        return False

    def sos_moment_matrix(clauses):
        n = max(abs(x) for x in sum(clauses, []))
        matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for assignment in itertools.product([-1, 1], repeat=n + 1):
            if all(assignment[var] * x <= 0 for x in clauses):
                count = sum(1 for var in range(-n, n + 1) if assignment[var] == 1)
                matrix[count][count] += 1
        return matrix

    def newton_basis(n):
        basis = [[0] * (n + 1) for _ in range(n + 1)]
        basis[0][0] = 1
        for k in range(1, n + 1):
            for j in range(k, -1, -1):
                if j > 0:
                    basis[k][j] = (k * basis[k - 1][j - 1] + basis[k - 1][j]) / (k + 1)
        return basis

    def schur_coefficients(matrix, basis):
        n = len(matrix) - 1
```

```
        coefficients = [0] * (n + 1)
        for i in range(n + 1):
            for j in range(n + 1):
                if matrix[i][j] != 0:
                    coefficients[i] += matrix[i][j] * basis[j][i]
        return coefficients

    def dominance(coefficients):
        max_coeff = max(coefficients)
        return coefficients.index(max_coeff) == len(coefficients) - 1

    n = random.randint(5, 40)
    clauses = generate_3sat_instance(n)
    satisfiable = is_satisfiable(clauses)
    matrix = sos_moment_matrix(clauses)
    basis = newton_basis(n)
    coefficients = schur_coefficients(matrix, basis)

    metric_value = dominance(coefficients)
    conjecture_holds = (satisfiable == metric_value)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Schur Positivity",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd7ddca9.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd7ddca9.py", line 69, in run_trial
    satisfiable = is_satisfiable(clauses)
                  ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd7ddca9.py", line 32, in is_satisfiabl
    if not any(assignment[var] * x <= 0 for x in clause):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd7ddca9.py", line 32, in <genexpr>
    if not any(assignment[var] * x <= 0 for x in clause):
                           ^^^
NameError: name 'var' is not defined. Did you mean: 'vars'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to NameError before producing results; cannot evaluate conjecture validity | next: Fix the NameError in the is_satisfiable function by using 'vars' instead of 'var' in the generator expression

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	54377
2	propose	ollama_remote	qwen3:8b	0	0	94663
3	propose	ollama_remote	qwen3:8b	0	0	113477
4	propose	ollama_remote	qwen3:8b	0	0	121117
5	preregistration	ollama_remote	qwen3:8b	0	0	27403
6	novelty	ollama_remote	qwen3:8b	0	0	23874
7	novelty	ollama_remote	qwen3:8b	0	0	18923
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17882
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12225
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9529
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11239
12	judge	ollama_remote	qwen3:8b	0	0	20782

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 525492 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/aa14d6657f34.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aa14d6657f34.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/aa14d6657f34.tar.gz` (if generated)